

Autonomous Chess-Playing Robot: Architecture, Perception, and Hardware Constraints

Canberk Yılmaz
Dept. of Computer Engineering
Izmir Institute of Technology
Izmir, Turkey
300201027

Vagif Aliyev
Dept. of Computer Engineering
Izmir Institute of Technology
Izmir, Turkey
300201131

Kaan Cesur
Dept. of Computer Engineering
Izmir Institute of Technology
Izmir, Turkey
290201066

Emir Efe Ulusoy
Dept. of Computer Engineering
Izmir Institute of Technology
Izmir, Turkey
300201103

Abstract—This report summarizes the development and final status of an autonomous chess-playing robotic system designed to play against a human opponent. The system architecture utilizes a "brain-on-PC" framework, integrating OpenCV for perception and custom Python scripts for kinematic control. While the Computer Vision pipeline achieved high robustness with a 92% square localization accuracy, the final integration phase was significantly impacted by hardware failures. A critical electrical fault occurred during simultaneous servo actuation, resulting in the loss of actuation hardware. Consequently, the project scope was adjusted to prioritize perception optimization and strict safety protocols. This paper details the system design, the software implementation of motion algorithms, and the engineering challenges related to power distribution.

Index Terms—Robotics, Computer Vision, Inverse Kinematics, Python, Fault Tolerance.

I. INTRODUCTION

The objective of this project was to design and build an autonomous system capable of playing physical chess against a human opponent. The functional requirements included perceiving the board state via a camera, making strategic decisions using the Stockfish chess engine [3], and executing moves using a 5-DOF robotic arm.

Initially proposed as a distributed ROS 2-based system, the project architecture was pivoted to a lightweight Python-centric framework to minimize communication latency [1]. This report details the implemented methodology, the specific software algorithms used for smoothing motion, and the results of the perception and control subsystems before the hardware failure.

II. METHODOLOGY

The system methodology integrates three modular Python subsystems: Mechanical, Perception, and Control.

A. Mechanical Setup

The mechanical design maintains a standard upright base-mounted configuration [1]. The robotic arm is right in front of

the opponent. To mitigate the torque load on the base servo, the workspace was centered to minimize full-extension reaches.

B. Perception Pipeline

Reliable board state estimation is critical. The vision system processes a video feed from an IP camera using the OpenCV library [2].

- **Perspective Transform:** Four corners of the board are detected to warp the view into a 640x640 pixel grid.
- **Occlusion Handling:** Vision processing is paused whenever the robot is in the "Moving" state.
- **Piece Detection:** The system utilizes HSV color thresholding to distinguish between physical pieces and shadows cast by the arm [1].

III. SOFTWARE IMPLEMENTATION DETAILS

A key contribution of this project is the custom "brain-on-PC" software stack that replaces heavy middleware. The following core functions drive the system:

A. Motion Smoothing (Arduino)

To address the mechanical oscillation observed in the upright arm, we implemented a custom smoothing function in the firmware. The `smoothMove` function replaces standard servo actuation with a linear interpolation algorithm:

- Instead of jumping instantly from angle A to B , the firmware divides the travel into discrete steps.
- A 15ms delay is introduced between steps to reduce the "jerk" (change in acceleration), significantly lowering the stress on plastic gears and preventing object slippage.

B. Inverse Kinematics Solver (Python)

We implemented a custom `inverse_kinematics` function in Python to map Cartesian coordinates (x, y, z) to joint angles.

- **Base Angle (θ_1):** Calculated using $\text{atan2}(y, x)$ to determine rotation in the horizontal plane.

- **Reachability Check:** The function calculates Euclidean distance d and verifies if $d < L_1 + L_2$. If the target is out of reach, the system throws a warning rather than stalling the motors.
- **Law of Cosines:** Used to determine the shoulder and elbow angles based on the triangle formed by the arm links L_1 and L_2 .

C. Communication Protocol

The Python controller communicates with the Arduino via a serial interface (115200 baud). A handshake protocol is defined where the Python script sends a `PING` command during initialization and waits for a `PONG` response to verify connection stability before attempting high-current movements.

IV. IMPLEMENTATION CHALLENGES & FAILURE ANALYSIS

The final phase of the project faced a critical hardware setback that altered the testing scope.

A. Actuation System Failure

During the integration of the full game loop, the team attempted to optimize the robot's speed by actuating two primary servo motors simultaneously. This operation caused a current spike exceeding the socket's rating, resulting in the thermal failure of the connection and the burnout of the base and shoulder servos.

B. Safety Constraints

Following the incident, the project was relocated to the department laboratory with strict safety protocols, including limiting operations to supervised hours (ending at 5:00 PM) and prohibiting simultaneous high-load actuation. This prevented the final "Full Game Loop" demonstration.

V. EXPERIMENTAL RESULTS

Despite the hardware failure, the individual subsystems were validated through unit testing during the mid-phase development.

A. Perception Robustness

The vision pipeline was stress-tested under varying ambient lighting. The perspective transform algorithm proved highly robust, successfully registering the board corners in 100% of trials. Piece occupancy detection achieved a 92% success rate [1]. The primary error source was dynamic shadowing, which was partially mitigated by the new HSV thresholding implementation.

VI. CONCLUSION

The transition to a Python-centric architecture successfully streamlined the software development process. The implementation of custom kinematic solvers and motion smoothing algorithms provided a viable alternative to complex middleware. However, the mechanical failure highlighted the critical need for industrial-grade power distribution when actuating multiple servos. Future work would prioritize electrical safety margins to enable the simultaneous joint movements required for fluid gameplay.

REFERENCES

- [1] Group 10, "Midphase Progress Report," CENG483, Izmir Institute of Technology, Dec. 2025.
- [2] OpenCV Team, "OpenCV: Open Source Computer Vision Library," 2025.
- [3] T. Romstad et al., "Stockfish Chess Engine," 2025.